Chapter 6 : Blazor

Er. Piyush Pant

Blazor

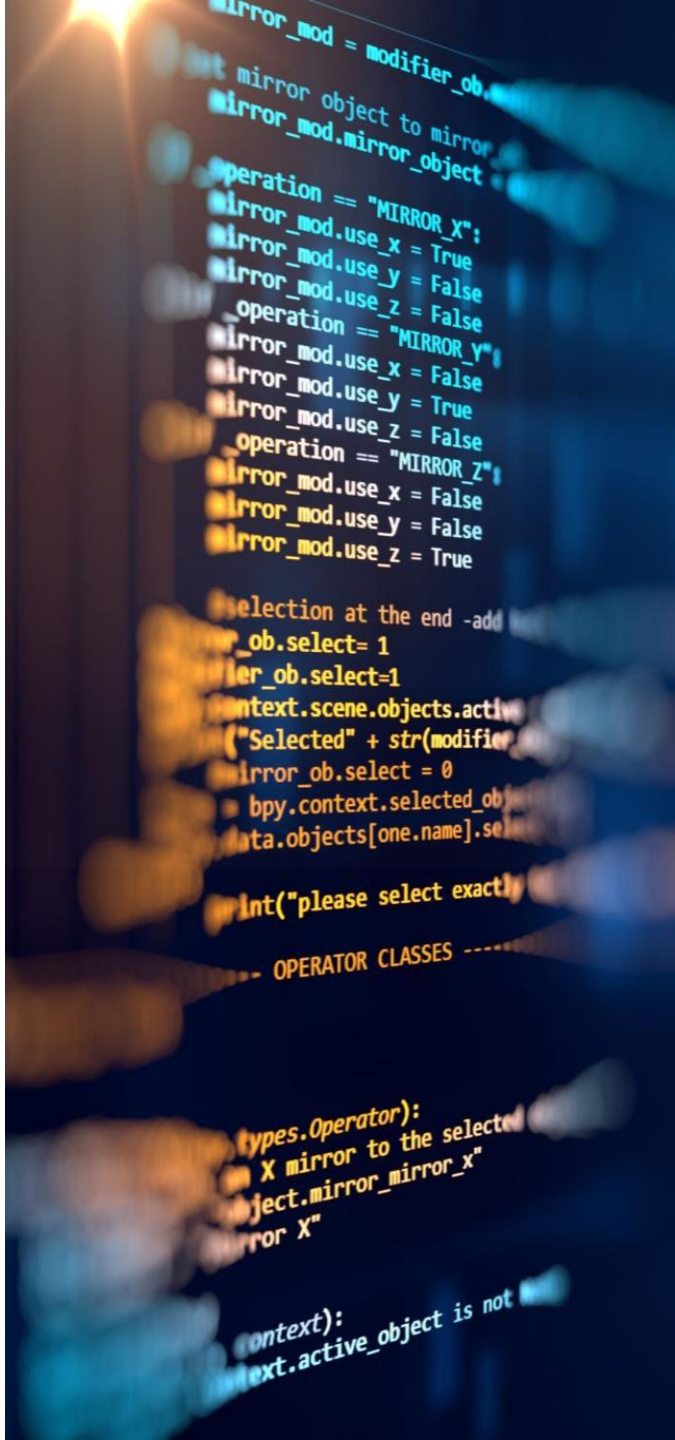
- Web development has traditionally required two languages: **C#** (or any backend language) for server-side logic and **JavaScript** for client-side interactivity. But what if you could build full-stack web applications using just C#?
- That's where Blazor comes in. Blazor is a modern framework from Microsoft that allows developers to build interactive web UIs using C#, Razor, and .NET without writing JavaScript for most scenarios.

Blazor

- Blazor is a Microsoft web framework that lets developers build interactive web applications using C# and .NET instead of JavaScript. It’s powerful for .NET developers but comes with trade-offs in performance, ecosystem maturity, and browser support.

Advantages of Blazor

- **C# for front-end**: Developers can use existing C# skills instead of learning JavaScript frameworks like React or Angular.
- **Code reusability**: Shared libraries and logic across client and server.
- **Integration with .NET ecosystem**: Seamless use of NuGet packages, Entity Framework, and other .NET tools.
- **Strong tooling**: Visual Studio and Azure integration provide a robust development environment.
- **Cross-platform**: Works on Windows, Linux, and macOS.
- **Rapid development**: Familiar syntax and reduced context switching between languages.
- **Future-proof**: Backed by Microsoft, with ongoing improvements and enterprise adoption.



Disadvantages of Blazor

- **Performance limitations**: Blazor WebAssembly apps can be slower than JavaScript frameworks, especially on initial load.
- **Large payload size**: WASM apps often have bigger download sizes, affecting load times.
- **Smaller ecosystem**: Compared to React, Angular, or Vue, Blazor has fewer third-party libraries and community resources.
- **Browser compatibility**: While modern browsers support WebAssembly, older ones may not.
- **Server dependency (Blazor Server)**: Requires constant connection; latency and scalability can be issues.
- **Learning curve for non-.NET developers**: Developers outside the Microsoft ecosystem may find it harder to adopt

Blazor Hosting Models

Blazor Server

Runs on the server; UI updates are sent to the browser via **SignalR**.

Advantages:

- Small initial download size (fast startup).
- Full access to server resources (e.g., databases, APIs).
- Centralized security and easier updates.

- **Disadvantages**:
  - Requires constant network connection.
  - Latency can affect responsiveness.
  - Scalability limited by server load.

  Best for: Enterprise/internal apps where users have reliable connections.

Blazor Hosting Models

Blazor WebAssembly (WASM)

Runs directly in the browser using **WebAssembly**;

no server dependency for UI logic.

Advantages:

- Works offline once loaded.
- Scales better since computation is client-side.
- No need for continuous server connection.

Disadvantages:

- Larger initial download (runtime + app).
- Slower startup compared to Blazor Server.
- Limited access to server resources unless APIs are exposed. **Best for**: Public-facing apps where offline support and scalability matter.

Blazor Hosting Models

Blazor Hybrid

Combines Blazor with native frameworks like **.NET MAUI** or **Electron** to run on desktop or mobile.

Advantages:

- Access to native device APIs (camera, sensors, etc.).
- Single codebase for web, desktop, and mobile.
- Rich UI with native performance.

Disadvantages:

- Larger app size.
- More complex deployment.
- Requires knowledge of native platforms.

Best for: Cross-platform apps needing both web and native features.

Feature	Blazor Server	Blazor WebAssembly	Blazor Hybrid
Execution	Server via SignalR	Browser via WebAssembly	Native desktop/mobile + Blazor
Startup Speed	Fast (small payload)	Slower (large payload)	Moderate (depends on platform)
Offline Support	❌ No	✅ Yes	✅ Yes
Scalability	Limited by server resources	Client-side scaling	Depends on native platform
Best Use Case	Enterprise apps, intranets	Public apps, offline scenarios	Cross-platform native apps

Blazor One way Binding

Data flows only from C# → UI.

The UI shows the value of a property, but user changes don't automatically update it.

```
@page "/oneway"
<h3>One-Way Binding Example</h3>
<p>Message: @message</p>
<button @onclick="ChangeMessage">Change Message</button>
@code {
    private string message = "Hello from Blazor!";
    private void ChangeMessage()
    {
        message = "Message updated!";
    }
}
```

Here, `@message` is displayed in the UI.

Clicking the button updates the property in C#, and the UI reflects it.

But if the user typed something in the UI, it wouldn't change `message` automatically.

Blazor Two Way Binding

Data flows both ways: C# ↔ UI.

The UI updates when the property changes, and the property updates when the user changes the UI (e.g., typing in an input box).

```
@page "/twoway"
<h3>Two-Way Binding Example</h3>
<p>Enter your name:</p>
<input @bind="name" />
<p>Hello, @name!</p>
@code {
    private string name = "Blazor User";
}
```

Here, typing in the input box updates the `name` property automatically, and the greeting updates in real time.

Event handling

- Event handling in Blazor is a way to make your web applications interactive by responding to user actions such as clicking buttons, typing in text boxes, or submitting forms.
- Instead of writing JavaScript, you write C# methods that run when these events happen in Blazor.
- When a user interacts with your app, Blazor "listens" for these events and calls the corresponding C# code you define.
- This allows your app to react immediately to user input, creating a dynamic and responsive experience.

Basic Event Handling

Syntax: `@on{event}="MethodName"`

```
<button @onclick="HandleClick">Click Me</button>

void HandleClick()

{

    Console.WriteLine("Button clicked!");

}
```

Common Events

Blazor supports many common events that help you respond to user interactions. Here are some of the most frequently used ones:

- `@onclick` → Triggered when a user clicks a button or any clickable element. Use this to handle clicks.
- `@oninput` → Fires every time the user types or changes the value inside an input field. Useful for live updates as the user types.
- `@onchange` → Occurs when the value of an input, dropdown, or checkbox changes and loses focus. Good for final value changes.
- `@onsubmit` → Happens when a form is submitted, often used to process form data.
- `@ondblclick` → Triggered on double-clicks.
- `@onmouseover` → Fires when the mouse pointer moves over an element.
- `@onkeydown` → Occurs when a key is pressed down.
- `@onkeyup` → Happens when a key is released.

Event Arguments

- Event arguments are used to get additional information about an event when it happens.
- For example, when a user types in an input box, the event argument can tell you what the new text is. When a mouse click happens, the event argument can tell you where the mouse was clicked or which button was pressed.
- In this example, the `OnInputChanged` method receives an event argument `e` that contains the new value typed by the user. You can then use this value in your code to update the UI or perform other actions.
- Event arguments are useful whenever you need to know more details about the event beyond just that it happened. They help make your app more interactive and responsive.

Some events provide arguments with useful data:

- **`ChangeEventArgs`** → Contains new value of input
- **`MouseEventArgs`** → Provides mouse position, button info

Example:

- ```
  <input @oninput="OnInputChanged" />

  @code {
   string message = "";
   void OnInputChanged(ChangeEventArgs e)
   {
   message = e.Value?.ToString();
   }
  }
  ```
  ```

Routing

Routing decides which component (page) should be displayed for a given URL.

In Blazor

- Each page is a Razor Component
- Routing is done using the @page directive

Directive

A directive is a keyword prefixes with @ that provides instructions to the Razor engine.

They control routing, layout ,dependency injection, rendering and component behavior.

Routing

Blazor routing is handled in App.razor

- `Router` → Finds matching routes.
- `Found` → Loads the component if the URL exists.
- `RouteView` → Renders the matched component inside the layout.
- `NotFound` → Handles unmatched routes (404-like behavior).

```
<Router AppAssembly="@typeof(Program).Assembly">
  <Found Context="routeData">
    <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
  </Found>
  <NotFound>
    <LayoutView Layout="@typeof(MainLayout)">
      <p>Sorry, there's nothing at this address.</p>
    </LayoutView>
  </NotFound>
</Router>
```

NavLink

- In Blazor, the **`NavLink`** component uses the `Match` property to determine how the current URL is matched against the link's `href`.
- You can choose between **`NavLinkMatch.All`** (exact match) and **`NavLinkMatch.Prefix`** (matches any URL that starts with the given prefix).

Key Details on `NavLink.Match`

- **Namespace & Assembly**: `Microsoft.AspNetCore.Components.Routing` (part of ASP.NET Core Blazor framework)
- **Purpose**: Controls whether a navigation link is considered *active* based on the current URL.
- **Values**:
  - `NavLinkMatch.All` → Active only when the current URL matches the link's `href` exactly.
  - `NavLinkMatch.Prefix` → Active when the current URL starts with the link's `href`

Example

```
<NavLink href="/" Match="NavLinkMatch.All">
    Home
</NavLink>

<NavLink href="/products" Match="NavLinkMatch.Prefix">
    Products
</NavLink>
```

In this example:

- The **Home** link is active only on `/`.
- The **Products** link is active on `/products`, `/products/details`, `/products/edit`, etc.